

Convex MPC controller (QP): what it does and why it is correct

July 2026

Summary. One convex QP per step (672 decision variables) solved by `quadprog` in about 0.9 s against the 900 s sampling budget. On the demo stressed day (24 h at 60 % flow capacity) it keeps every consumer at 100.000 % demand met, where a hold-nominal baseline falls to 82.2 % at its worst consumer (90.7 % across the five). Feasible on 96/96 steps. The severe 35 % benchmark day (KPC 97.56 %) is in comparison_validation. One logged QP step is re-verified independently of the solver (feasibility residuals, the KKT conditions, an active-set re-solve). All seven validator invariants pass (Section 4).

1 What this component does and what I did

This is the Koopman predictive controller (KPC): one sparse convex QP per 15-minute sample ($T_s = 900$ s) for the 23-node, 29-edge network with five consumers. The input sequence U stacks the source supply command $T^{(0,s)}$, the per-edge flow references r_q and the consumer return commands $T^{(i,r)}$. The direct multi-step maps from `predictor_open_loop/` make predicted delivered heat affine in U ,

$$c_{\text{pred}} = c_p (F z_0 + G U + G_d D), \quad (1)$$

with z_0 the 47-feature lifted state and D the demand forecast, both known at solve time. Stacked over $N_p = 12$ horizon steps, c_{pred} is length-60 (one delivered heat per consumer per step). F is 60×47 , G is 60×420 (block-lower-triangular in the horizon), G_d is 60×60 . $G_d D$ folds into a constant, so the problem stays a convex QP in U ; no rollout happens inside the optimisation.

The decision vector is $\omega = [U; s_{\text{hi}}; s_{\text{lo}}; s_{\text{ord}}; s_{\text{mix}}]$ with the four s blocks the soft-constraint slacks. The cost rewards delivered heat, penalises the linearised source energy $\alpha (T^{(0,s)} - T^{(0,r)}) Q_{\text{net}}$, and penalises the input increments (the only cost-shaping quadratic term, which makes the U -block of the Hessian strictly convex), plus ℓ_1 slack prices and a 10^{-9} ridge:

$$\begin{aligned} J(\omega) = & \underbrace{-c_p \mathbf{1}^\top G U}_{\text{delivery reward}} + \underbrace{\alpha (T^{(0,s)} - T^{(0,r)}) Q_{\text{net}}}_{\text{source energy (lin.)}} + \underbrace{\Delta U^\top W \Delta U}_{\text{input rate}} \\ & + \underbrace{\rho^\top s}_{\text{slack prices}} + 10^{-9} \|\omega\|^2, \quad \Delta U = D_U U + D_{\text{pre}} u_{\text{prev}}. \end{aligned} \quad (2)$$

The slack prices are unequal: $\rho_{\text{hi}} = \rho_{\text{slack}}$ on the demand cap, $\rho_{\text{lo}} = \rho_{\text{ord}} = 10 \rho_{\text{slack}}$ because $c < 0$ or $T^{(i,r)} > T^{(i,s)}$ is unphysical.

The QP has four soft and four hard constraint families.

Soft (priced slacks, so the QP stays feasible):

- demand contract $c_{\text{pred}} \leq d + s_{\text{hi}}$;
- non-negative heat $c_{\text{pred}} \geq -s_{\text{lo}}$;

- return below supply $T^{(i,r)} \leq T^{(i,s),\text{pred}} + s_{\text{ord}}$ (the hard form is infeasible in low-demand windows; the substation enforces the bound anyway);
- junction mixing residual $|g_{\text{mix}}(z_0, U)| \leq s_{\text{mix}}$.

Hard (`kpc_v2_solve.m`):

- actuator boxes: $T^{(0,s)} \in [14, 26]^\circ\text{C}$ (lower bound raised to the tuned 17.69°C), r_q within $[0.5694, 1.5] \times \text{nominal}$ (in-code default 0.3), $T^{(i,r)} \in [15, 26]^\circ\text{C}$;
- rate caps $|u_k - u_{k-1}| \leq (7.5 \text{ K}, 4.5 \text{ kg/s}, 15.0 \text{ K})$ per step;
- the demand-adequacy floor (3);
- per-node Kirchhoff mass balance (4) on the flow references, $13 + 13 = 26$ rows per horizon step.

$$r_q^{(i)}(h) \geq \min\left(s_{\text{adeq}} \frac{d^{(i)}(h)}{c_p \Delta T_{\text{floor}}}, 0.95 r_{q,\text{max}}^{(i)}\right), \quad s_{\text{adeq}} = 1.06, \quad \Delta T_{\text{floor}} = T^{(0,s),\text{nom}} - T_{\text{min}}^{(r)}, \quad (3)$$

with $T^{(0,s),\text{nom}} = 20^\circ\text{C}$ and $T_{\text{min}}^{(r)} = 15^\circ\text{C}$, and

$$M_s r_{q,h} = 0, \quad M_r r_{q,h} = 0 \quad \forall h. \quad (4)$$

M_s and M_r are the supply and return incidence blocks with one row dropped from each (`build_incidence_v2.m`): the $F0$ row carries no source injection term, so keeping it would pin the trunk reference at zero, and the $R0$ row is the only one holding the loop-closure edge. So the setpoints are conservative at every interior node, and the trunk and loop-closure references are set by the box and rate limits instead of by this block. The realised flow then follows the setpoints at steady state under the per-edge lag $q_{k+1} = a q_k + (1 - a) r_{q,k}$, which is why the flow-balance residual in Section 3 is a steady-state statement and not a per-step identity.

The outer loop (`kpc_step_loop.m`) applies only the first input block; of it only $T^{(0,s)}$ and the flow references reach the plant. The $T^{(i,r)}$ entries are a predictor degree of freedom, and the substations set the physical returns. If `quadprog` fails, the loop holds the previous plan; over the 90-day run this fired once in 8640 steps. The tune is locked in the three committed artefacts of Table 1.

knob	value	selected by
N_p (prediction horizon, steps)	12	horizon sweep
N_c (control horizon: one move, held over N_p)	1	cross-scenario sweep (Occam pick)
α	1	two-pass Pareto
ρ_{slack}	1.195	Bayesian optimisation
W input-rate penalty ($T^{(0,s)}/r_q/T^{(i,r)}$)	984 / 1193 / 186	Bayesian optimisation
s_{adeq}	1.06	demand-adequacy floor
$T^{(0,s)}$ floor	17.69°C	part of the locked tune (<code>best_tune.mat</code>)
r_q lower factor	$0.5694 \times \text{nominal}$	part of the locked tune (<code>best_tune.mat</code>)
rate caps	7.5 K / 4.5 kg/s / 15.0 K	verified (box bounds bind at stress; rate cap rarely)
QP algorithm	interior-point-convex	sensitivity sweep

Table 1: The locked tune, stored in `best_horizons.mat`, `best_alpha.mat` and `best_tune.mat`.

The tune was selected once, by the sweeps named in the third column, and then frozen: the three `.mat` files above are the record of that selection and every runner in this package reads its constants from them rather than hard-coding them. The selection scripts themselves are not part of this package, so the numbers arrive here as given values. What can be checked here is that they are used consistently everywhere and that the result does not hinge on a lucky setting, which is what the horizon sweep in `comparison_validation.pdf` and the stress sweep in `robustness_validation.pdf` are for.

Two baselines run in the identical envelope: a hold-nominal schedule and a demand-following rule-based controller; the iterated one-step Koopman-LMPC is compared separately in `comparison_validation`. Demand met is a daily energy ratio per consumer. At design capacity KPC meets 100 % on all 30 (consumer, scenario) cells. At stressed capacity (`mdot_scale` = 0.6, i.e. 60 % of nominal flow) it holds 100.000 % at every consumer, where hold-nominal falls to 82.206 % at its worst consumer (+17.79 pp on that consumer; its suite mean is 90.679 %). All 96 QP solves succeed, at 923 ms mean and 1619 ms max, about 0.10 % of the sample budget.

Code layout

file	role
<code>kpc/build_kpc_v2_matrices.m, truncate_pred_to_Np.m</code>	stack the fitted maps into F, G, G_d at N_p
<code>kpc/kpc_v2_solve.m</code>	assembles and solves the QP of Section 1 (one function)
<code>kpc/kpc_step_loop.m</code>	the receding-horizon loop: apply first move, step plant, repeat
<code>kpc/build_incidence_v2.m</code>	supply/return incidence behind the Kirchhoff rows
<code>lmpc/</code>	the iterated one-step baseline in the identical envelope
<code>run_rbc_demand_following.m</code>	the rule-based baseline
<code>demo_controller.m, run_scenario_suite.m</code>	the demos below
<code>tests/validate_controller.m, tests/validate_qp.m</code>	the coded checks of Section 4

2 How to reproduce

From the repo root in MATLAB (R2024b with the Optimization Toolbox), run `startup; demo_controller` (about 5 min, 96 QP solves). It runs the stressed 24 h loop plus hold-nominal and writes `results/demo.mat` and `figures/tracking.pdf`. The 30-minute warmup only seeds the two past samples the lift’s delay features need; `build_plant` fills the pipes at the nominal 20 °C, so the warmup does not heat the network. `validate_controller` asserts T1–T7 on the saved data, `validate_qp` runs Q1–Q3 on the rebuilt mid-day QP, and `run_all_tests` runs all five repo validators. The figures in this document are redrawn by `plot_flow_balance.m` in this folder and by `viz_controller_signals.m` and `viz_controller_tracking.m` in `visualizations/3_controller`; each loads the saved results and needs no re-simulation.

3 The figures

3.1 Demand tracking at the hardest consumer (figures/tracking.pdf)

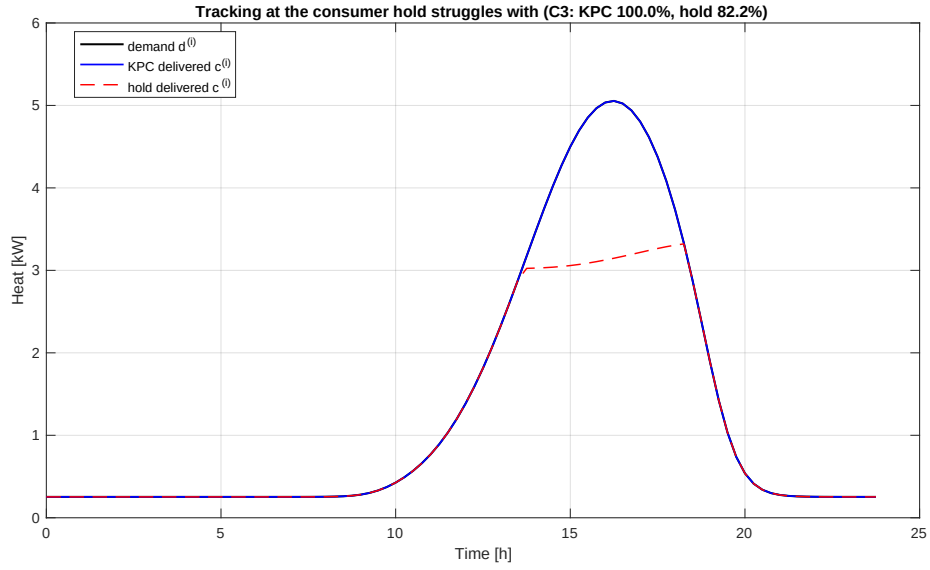


Figure 1: Demand and delivered heat at the consumer where hold-nominal fails worst, over the stressed 24h day. The KPC trace sits on the demand curve; the hold trace falls away at the peaks.

At 60 % flow capacity a fixed nominal schedule cannot carry the peak load. The controller raises the flow references toward the box, helped by the adequacy floor (3), while the demand contract $c \leq d$ keeps the trace from overshooting. A failure would be the KPC trace detaching from the demand curve at the peak.

3.2 Thermal constraints and solve time on the hard day (../visualizations/3_controller/viz_controller_thermal_hard.pdf)

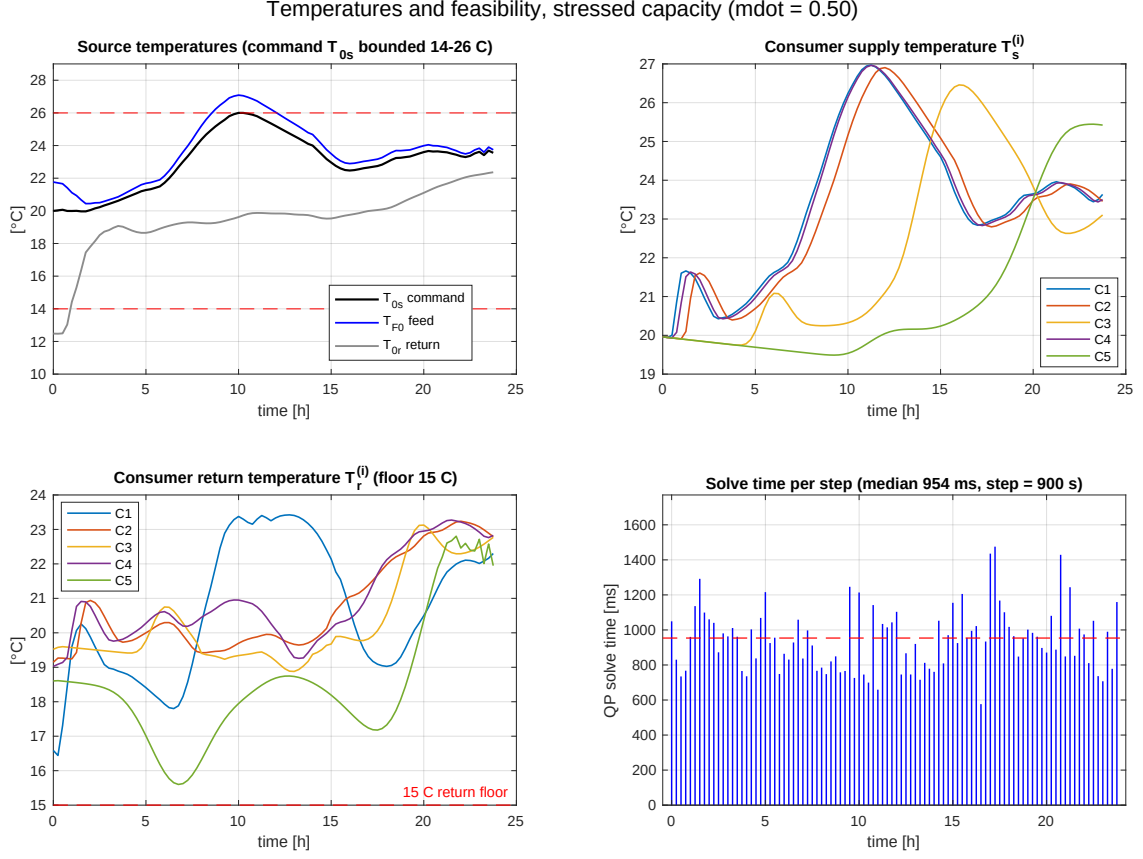


Figure 2: Thermal constraints and QP solve time on the hard day ($\dot{m} = 0.50$). The supply command touches its 26°C cap at the demand peak and stays 1 to 3K below it the rest of the day; returns approach but never cross the 15°C floor; every solve sits orders of magnitude under budget.

These two constraint figures use a tighter half-capacity run ($\dot{m} = 0.50$) than the 0.60 demo and validator above, so the bounds bite harder. Two honest readings. First, the dashed bounds hold the command $T^{(0,s)}$; the source feed and the delayed consumer supplies behind it can peak slightly above 26°C because warm return water blends through the source. That is plant behaviour, not a violation. Second, plan and plant agree at the return floor: the substation clips extraction at 15°C, and the QP plans with the same hard bound on its $T^{(i,r)}$ variable (never applied to the plant). Failures would be a command above the cap, a return below the floor, or solve times drifting toward the budget.

3.3 Hydraulic actuation on the hard day

(../visualizations/3_controller/viz_controller_hydraulic_hard.pdf)

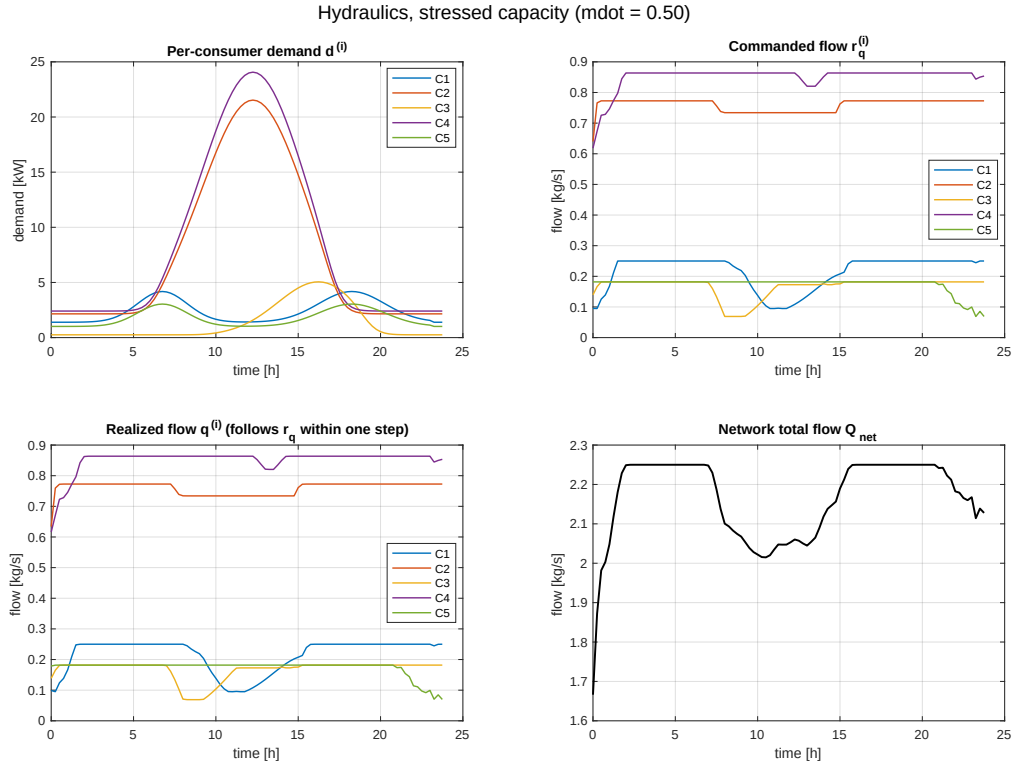


Figure 3: Hydraulic actuation on the hard day: the large consumers sit at their per-edge flow cap for most of the day rather than following the demand shape, which is what running out of hydraulic headroom looks like; realised edge flows follow the commands within about one step.

The large consumers C2 and C4 ride their caps for most of the half-capacity day. The total-flow panel (Q_{net} , by Kirchhoff the sum of the consumer flows) plateaus where the per-consumer flows cap together. Once the flows are capped, the supply temperature is the only lever left, which is why it is pushed to its own cap at the peak. A failure would be a commanded flow outside the box or rate cap, or realised flow diverging from its command.

3.4 Kirchhoff balance of the applied commands (figures/flow_balance.pdf)

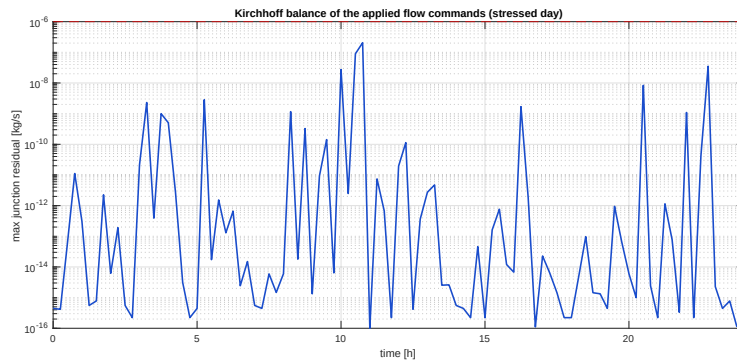


Figure 4: Worst junction mass-balance residual per step of the applied flow commands, checked from the outside by re-applying the incidence matrices.

The equality (4) lives inside the optimiser, so it is checked independently from the outside. The worst residual stays below 2.1×10^{-7} kg/s: below the 10^{-6} kg/s negligibility line and six or more orders under the ~ 0.1 kg/s edge flows. This is solver tolerance, what a hard equality leaves behind. A failure would be a residual near the flow scale.

4 The automated checks

`tests/validate_controller.m` loads the saved `demo.mat` and asserts seven invariants; it refits nothing and errors on the first failure.

#	invariant	bar	result
T1	every QP solve feasible	exitflag > 0	96/96 steps
T2	met % on every design cell	≥ 99.99	30/30 at 100
T3	stressed suite-mean met %	$\geq 95\%$	100.000 %
T4	worst-consumer vs hold-nominal	beats it	100.000 vs 82.206 % (+17.79 pp)
T5	$c \leq d$ on the realised trace	$\max(c - d) \approx 0$	1.5×10^{-11} W
T6	applied r_q in box and rate limit	inside	holds; box binding
T7	mean solve time	$\leq 5\% T_s$	923 ms mean, 1619 ms max

Table 2: The seven closed-loop invariants of `validate_controller.m`.

T1: the priced slacks recover feasibility by construction where a hard-constrained form would be infeasible (Kerrigan and Maciejowski, 2000). T2–T4 are the performance claims; hold-nominal runs on the identical throttled plant and warmup, so the margin is not a weakened-baseline artefact. T5 is scoped honestly: the plant clips delivered heat at the substation, so it confirms closed-loop bookkeeping rather than controller-side enforcement.

The numeric bars are acceptance floors that encode *usable*, not *optimal*: T2’s 99.99 is a near-exact full-delivery floor, T3’s 95 % and T7’s 5 % T_s are deliberately loose (the run reaches 100 % and clears speed by a factor-50 margin), and T5 is an exact bookkeeping identity at machine precision, not a tunable bar.

`tests/validate_qp.m` asks whether the reported optimum is a genuine solution of the stated QP, independent of the solver’s exit flag. It rebuilds the mid-day QP from the exported matrices and multipliers and checks, at 10^{-6} , primal feasibility (Q1) and the full KKT conditions (Q2, multipliers nonnegative to -10^{-9}). For this convex QP these are necessary and sufficient for the global optimum (Boyd and Vandenberghe, 2004). An active-set re-solve then reproduces the objective to 10^{-6} relative and the input block U to 10^{-3} (Q3; U only, because the input-rate penalty makes the Hessian positive definite there, so U is the part of the solution that is unique). Passing all three means the point `quadprog` returns for this step is a certified global optimum of exactly the problem in Section 1. It is a check on the solver and on the matrices, run on one representative mid-day step: it re-solves that step rather than replaying the whole closed loop, so it certifies the QP the controller poses, not every input the controller applied.